

Normalization of the ERD

1NF: Atomic Values

The first database normalization step is First Normal Form (1NF). In this step, each cell should only contain one atomic value, meaning that multiple values within one cell are not allowed. To achieve this, the attributes with multiple values can be separated into individual rows in related tables. The following example shows a 1NF violation:

Before 1NF, with example data:

Fraud_incident_ID	Person_ID	Fraud_type	Communication_channel	Report_date
FI1	P1	Phishing	Email, Phone	04-07-2022
FI2	P2	Phishing, Identity theft	Social Media	16-02-2024
FI3	P3	Investment scam	Email	23-10-2025

In the table above two cells contain more than one value. For FI1, the attribute Communication_channel contains both Email and Phone. FI2 also has an 1NF violation, as Fraud_type consists of Phishing and Identity Theft. Therefore, the table does not satisfy 1NF as the values are not atomic. To solve this problem, the values could be separated into individual rows in relationship tables.

After 1NF, with example data:

Fraud type table:

Fraud_incident_ID	Fraud_type
FI1	Phishing
FI2	Phishing
FI2	Identity Theft
FI3	Investment Scam

Communication channel table:

Fraud_incident_ID	Communication_channel
FI1	Email
FI1	Phone
FI2	Social Media
FI3	Email

After these adaptations, each cell contains only one value, meaning that the attributes are atomic and the tables meet the 1NF requirements.

2NF: No Partial Dependencies

In our Report table, we need two columns together to know exactly which row we're looking at: Fraud_incident and Organisation_ID. That is because one incident can be reported to several organisations, so neither column alone is enough to point to one specific row, you need both.

Now imagine Organisation_name and Organisation_type was also written directly into the Report table. Those two things do not care which incident it was, they only belong to the organisation. So every time the same organisation showed up in a new report, we'd be typing its name in again, even though nothing about the organisation itself had changed. So if an organisation's name ever changed, we'd have to go find and fix it in every single row it appeared in.

Before, with example data:

Fraud_incident_ID	Organisation_ID	Organisation_name	Organisation_type	Report_date
1	O1	Police Netherlands	Police	2026-01-11
2	O1	Police Netherlands	Police	2026-01-16
1	O2	ABN AMRO	Bank	2026-01-11

The problem we want to avoid in the example above, is the repeated information. In this case, the Police Netherlands/Police that appears twice.

After, with example data:

Organisation table:

Organisation_ID	Organisation_name	Organisation_type
O1	Police Netherlands	Police
O2	ABN AMRO	Bank

Report Table:

Report_ID	Fraud_incident_ID	Organisation_ID	Report_date
R1	1	O1	2026-01-11
R2	2	O1	2026-01-16
R3	1	O2	2026-01-11

Now each organisation's name and type is written down just once, in its own table. The Report table just points to it using Organisation_ID, instead of repeating the details every time.

3NF: No Transitive Dependencies

The third step of database normalization is Third Normal Form (3NF). A database is in 3NF when it already meets the requirements of 1NF and 2NF, and when non-key attributes do not depend on other non-key attributes. Instead, each non-key attribute should depend directly on the primary key.

In our database, a transitive dependency could occur if information about the fraud type is stored directly in the Fraud_Incident table. For example, the table could contain both *Fraud_type_ID* and *Fraud_kind*. The problem is that *Fraud_kind* does not depend directly on *Fraud_incident_ID*. Instead, it depends on *Fraud_type_ID*, because each fraud type has one corresponding fraud kind.

This can be shown as:

Fraud_incident_ID → Fraud_type_ID → Fraud_kind

For example, if multiple fraud incidents are classified as phishing, the value "Phishing" would be repeated for every incident. This creates unnecessary duplicate data and could lead to inconsistencies. If information about the fraud type needed to be changed, it would need to be changed in multiple rows.

Before 3NF:

Fraud_incident_ID	Person_ID	Fraud_type_ID	Fraud_kind
FI1	P1	FT1	Phishing
FI2	P2	FT1	Phishing
FI3	P3	FT2	Identity Theft
FI4	P1	FT1	Phishing

The problem is that *Fraud_kind* depends on *Fraud_type_ID* instead of directly on *Fraud_incident_ID*. The value "Phishing" is also repeated multiple times.

After 3NF:

To solve this problem, we created a separate *Fraud_Type* table.

Fraud_Type

Fraud_type_ID	Fraud_kind
FT1	Phishing
FT2	Identity Theft

Fraud_Incident

Fraud_incident_ID	Person_ID	Fraud_type_ID
FI1	P1	FT1
FI2	P2	FT1
FI3	P3	FT2
FI4	P1	FT1

After this change, each fraud type is stored only once in the *Fraud_Type* table. The *Fraud_Incident* table only stores the *Fraud_type_ID* as a foreign key to link each incident to its type.

This removes the transitive dependency and reduces duplicated data. It also makes the database easier to maintain, because information about a fraud type only needs to be updated once. Therefore, our database satisfies the requirements of Third Normal Form (3NF).